

Chapter 53. Deep Learning in Depth

Networks from first principles: one training step by hand, convolution, a defect detector with a priced threshold, attention, and quantization.

Draft chapter

Contents

Chapter 53. Deep Learning in Depth

Why this matters

In plain English

53.1 From regression to a network

53.2 Training, worked by hand

53.3 The techniques that make training work

53.4 The architectures, and what each is for

53.5 Convolution, step by step

53.6 The project: defect detection on Riverstone's moulding line

53.7 Attention, step by step

53.8 Why transformers took over

53.9 Quantization and compression

53.10 When not to use deep learning

Common mistakes and how to spot them

In the real world: the model that watched the wrong thing

Tools

The project: a defect detector with an operating point

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 53. Deep Learning in Depth

Part VI — Production ML, Generative AI & MLOps

Chapter at a glance

You will learn to: describe a neural network as a stack of very simple parts · follow one training step by hand, from forward pass to updated weights · use the techniques that make training work (normalization, initialization, learning-rate schedules, dropout, early stopping) · say what convolution does to an image, by computing one · build and judge a defect-detection model for Riverstone's moulding line, choosing its threshold from the cost of each kind of mistake · work through a transformer's attention step by step on four tokens · explain quantization, pruning, and distillation, and measure what quantization costs and saves · and say when deep learning is the wrong tool.

Before you start: Chapter 19 (regression), Chapters 37 and 38 (how models are trained and evaluated: train/test splits, overfitting, precision and recall), Chapter 17 (NumPy and pandas), Chapter 29 (writing code you can test).

Time needed: 14–18 hours, spread over three weeks.

Tools: Python 3.12, NumPy, scikit-learn, matplotlib. Every calculation in this chapter runs on a laptop in seconds, with no GPU, no downloads, and no accounts.

Practice data: `generate_defect_images.py` in the companion folder builds 6,000 images of moulded lids on Riverstone's conveyor, 7.9% of them defective, with three defect types. Fixed seed, so your numbers match the book's.

Why this matters

Deep learning is the engine behind the things your business has started asking for: reading a photograph of a part and saying whether it's good, reading an email and extracting the order, answering a question from a manual. Chapters 54 to 58 build those applications. This chapter is what's inside them.

You don't need to be able to derive backpropagation to do that work well. You do need to be able to:

- **Say what the model is actually doing**, when a manager asks why it flagged a good part.
- **Recognize the failure modes:** a model that predicts the majority class and scores 92% accuracy, a model that learned the lighting rather than the defect, a model that works in the lab and not on the line.
- **Choose the operating point.** A defect detector's threshold is a business decision about the cost of a missed defect against the cost of a false alarm, and the data scientist is the only person in the room who can price it.

- **Read the vocabulary** in papers, vendor decks, and interviews: layers, activations, gradients, attention, quantization.

This chapter builds all of that from arithmetic you can check with a calculator.

In plain English

A neural network is a long chain of small, dumb decisions, tuned by trial and error.

Picture a factory line of inspectors, each with one very simple rule. The first looks at raw pixels and reports things like "it's brighter here than there". The second takes those reports and says "that pattern looks like an edge". The third: "those edges form a ring". The last one says "ring present, no hole: good part".

Nobody wrote those rules. The line starts with every inspector guessing at random, and after each part the factory is told whether it got the answer right. The blame is then passed backwards down the line: each inspector nudges their rule a little in the direction that would have made the answer better. That's **backpropagation**, and the size of the nudge is the **learning rate**.

Do that a few hundred thousand times and the early inspectors end up noticing edges, the middle ones shapes, and the last ones the decision. **Depth** is what gives the chain its power: simple parts, stacked, can express complicated things.

Two more ideas from this chapter fit the same picture. **Convolution** is one inspector sliding the same small rule across the whole image, so a scratch is found wherever it appears. **Attention**, the idea behind every large language model, is an inspector who, before deciding about one word, looks back at every other word and weighs how much each one matters.

53.1 From regression to a network

Chapter 19's linear regression computes one number from its inputs:

$$\text{prediction} = w_1x_1 + w_2x_2 + \dots + b$$

A **neuron** is exactly that, plus one twist: the result is passed through an **activation function** that bends it. Without the bend, stacking layers would be pointless, because a chain of linear functions is just another linear function. With it, a stack can describe curves, corners, and combinations.

```

import numpy as np

def relu(x):
    """The most common activation: keep positives, flatten negatives to zero."""
    return np.maximum(0, x)

weights = np.array([0.6, -0.4])      # one weight per input
bias = 0.1
inputs = np.array([2.0, 3.0])

weighted_sum = float(inputs @ weights + bias)
output = float(relu(weighted_sum))
print(f"inputs      {inputs}")
print(f"weighted sum {weighted_sum:.2f}    (2.0*0.6 + 3.0*-0.4 + 0.1)")
print(f"after ReLU    {output:.2f}")

negative_case = float(relu(np.array([1.0, 3.0]) @ weights + bias))
print(f"a case that comes out negative: ReLU gives {negative_case:.2f}")

```

```

inputs      [2. 3.]
weighted sum 0.10 (2.0*0.6 + 3.0*-0.4 + 0.1)
after ReLU   0.10
a case that comes out negative: ReLU gives 0.00

```

Line by line:

- `import numpy as np` brings in NumPy, which does arithmetic on whole arrays at once. `np` is the conventional short name.
- `def relu(x):` defines the activation. `np.maximum(0, x)` compares each element of `x` with 0 and keeps the larger, so -2.3 becomes 0 and 1.7 stays 1.7. It is the whole function: no curve, no exponentials, which is why it's fast.
- `weights = np.array([0.6, -0.4])` is the neuron's two weights, one per input. These are the numbers training will change.
- `bias = 0.1` shifts the result up or down regardless of the inputs, exactly like the intercept in Chapter 19's regression.
- `inputs @ weights` is the **dot product**: $2.0 \times 0.6 + 3.0 \times -0.4$. The `@` operator means matrix multiplication in NumPy, and for two one-dimensional arrays that is the sum of the products.
- `float(...)` converts NumPy's single-element result into an ordinary Python number, so the f-string prints `-0.1` rather than `np.float32(-0.1)`.
- The last two lines show the bend: a weighted sum of -0.9 comes out of ReLU as 0. That flattening is what lets a stack of neurons make decisions rather than straight lines.

Layers. A **layer** is a row of neurons all looking at the same inputs, and a network is layers in sequence: the outputs of one become the inputs of the next. Three terms follow from that:

Term	Means
Input layer	the raw numbers: 1,024 pixel values for a 32×32 image
Hidden layer	any layer between input and output; "deep" means more than one

Term	Means
Output layer	the answer: one number for "probability this part is defective", ten for "which digit is this"
Parameters	every weight and bias, all learned. A $1,024 \rightarrow 64 \rightarrow 1$ network has $1,024 \times 64 + 64 + 64 + 1 = 65,665$ of them

That last number is worth holding on to. A model is a big pile of numbers, and training is the search for a good pile.

53.2 Training, worked by hand

Training repeats four steps: **predict, measure the error, work out which way each weight should move, move it a little**. Here is all four, on a network small enough to check with a calculator: two inputs, two hidden neurons, one output.

```
import numpy as np

x = np.array([1.0, 2.0])          # one training example: two features
y_true = 1.0                     # the right answer for it

W1 = np.array([[0.5, -0.3],      # hidden layer: 2 inputs -> 2 neurons
               [0.8, 0.2]])
b1 = np.array([0.0, 0.1])
W2 = np.array([0.7, -0.5])       # output layer: 2 hidden -> 1 output
b2 = 0.05

# --- forward pass: compute the prediction
z1 = x @ W1 + b1                 # weighted sums for the two hidden neurons
a1 = np.maximum(0, z1)           # ReLU
z2 = float(a1 @ W2 + b2)         # weighted sum for the output neuron
loss = (z2 - y_true) ** 2        # squared error: how wrong we were

print(f"hidden sums z1 = {np.round(z1, 3)}")
print(f"after ReLU a1 = {np.round(a1, 3)}")
print(f"prediction z2 = {z2:.3f} true = {y_true}")
print(f"loss = {loss:.3f}")
```

```
hidden sums z1 = [2.1 0.2]
after ReLU a1 = [2.1 0.2]
prediction z2 = 1.420 true = 1.0
loss = 0.176
```

Line by line:

- `x` is one example, and `y_true` the answer we want. Real training uses thousands; the arithmetic is identical.
- `W1` is a 2×2 matrix: **row *i*, column *j*** is the weight from input *i* to hidden neuron *j*. `b1` holds one bias per hidden neuron.
- `x @ W1` multiplies the input vector by the matrix, giving one weighted sum per hidden neuron: for neuron 0 that's $1.0 \times 0.5 + 2.0 \times 0.8 = 2.1$, then $+ b1[0] = 0.0$, so 2.1.

- `np.maximum(0, z1)` applies ReLU to both hidden sums at once. Neuron 1's sum is 0.1, which survives; nothing is negative here, so `a1` equals `z1` this time.
- `a1 @ W2 + b2` is the output neuron. `float(...)` again just makes printing tidy.
- `(z2 - y_true) ** 2` is the **loss**: the squared difference between prediction and truth. Squaring makes every error positive and punishes big misses more than small ones. Training exists to make this number smaller.

Now the part people find mysterious, which is really the chain rule from school calculus. **The gradient of the loss with respect to a weight answers one question: if I increase this weight by a tiny amount, does the loss go up or down, and how fast?**

```
# --- backward pass: how should each weight change?
dloss_dz2 = 2 * (z2 - y_true)          # derivative of (z2 - y)^2 with respect to z2
dW2 = dloss_dz2 * a1                   # each output weight is blamed in proportion to its input
db2 = dloss_dz2

da1 = dloss_dz2 * W2                   # blame passed back to the hidden activations
dz1 = da1 * (z1 > 0)                   # ReLU passes blame only where it was active
dW1 = np.outer(x, dz1)                 # each hidden weight is blamed in proportion to its input
db1 = dz1

print(f"dloss/dz2 = {dloss_dz2:.3f}")
print(f"dW2      = {np.round(dW2, 3)}      db2 = {db2:.3f}")
print(f"dz1      = {np.round(dz1, 3)}")
print(f"dW1      =\n{np.round(dW1, 3)}")
```

```
dloss/dz2 = 0.840
dW2      = [1.764 0.168]      db2 = 0.840
dz1      = [ 0.588 -0.42 ]
dW1      =
[[ 0.588 -0.42 ]
 [ 1.176 -0.84 ]]
```

Line by line:

- `dloss_dz2 = 2 * (z2 - y_true)`: differentiating $(z2 - y)^2$ gives $2(z2 - y)$. It's positive when we predicted too high, so the update will push the prediction down.
- `dW2 = dloss_dz2 * a1`: the output is `a1[0]*W2[0] + a1[1]*W2[1] + b2`, so changing `W2[0]` changes the output in proportion to `a1[0]`. A weight whose input was large gets most of the blame.
- `db2 = dloss_dz2`: the bias is added directly, so its share is the error itself.
- `da1 = dloss_dz2 * W2` sends the blame backwards: a hidden neuron connected by a large weight contributed more, so it receives more.
- `dz1 = da1 * (z1 > 0)` is ReLU's derivative. `(z1 > 0)` is a boolean array, which NumPy treats as 1 and 0: a neuron that was switched off contributed nothing, so it gets no blame. (This is why "dead" ReLU neurons, permanently negative, stop learning.)
- `np.outer(x, dz1)` builds the 2×2 matrix of blame for `W1`: element (i, j) is `x[i] * dz1[j]`, the same "in proportion to your input" rule one layer down.

Then the update, which is one line and the whole of **gradient descent**:

```

learning_rate = 0.05
W1_new = W1 - learning_rate * dW1
b1_new = b1 - learning_rate * db1
W2_new = W2 - learning_rate * dW2
b2_new = b2 - learning_rate * db2

z1n = np.maximum(0, x @ W1_new + b1_new)
z2n = float(z1n @ W2_new + b2_new)
print(f"prediction before {z2:.3f}, after one step {z2n:.3f}, target {y_true}")
print(f"loss      before {loss:.3f}, after one step {(z2n - y_true) ** 2:.3f}")

```

```

prediction before 1.420, after one step 1.019, target 1.0
loss      before 0.176, after one step 0.000

```

Line by line: each weight moves *against* its gradient, because the gradient points uphill and we want to go down. `learning_rate` decides how big the step is: too small and training crawls, too large and it overshoots and bounces. That is the entire algorithm. A real network does this with millions of weights and batches of examples, on a GPU, using better optimizers (**Adam** adapts the step size per weight and is the usual default), but every one of those steps is the arithmetic above.

One training step, with the numbers from section 53.2

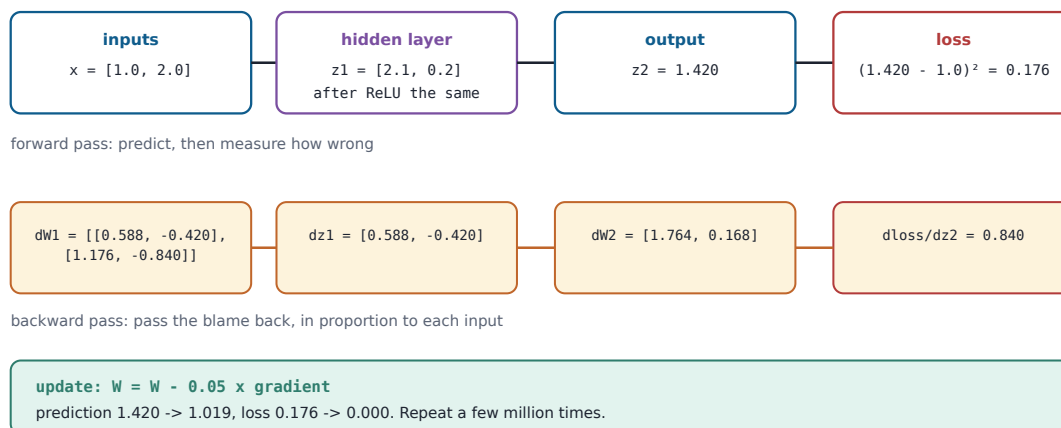


Figure 53.2 – One training step, with section 53.2's numbers. Everything else is this, repeated.

Vocabulary you now own:

Term	Means
Forward pass	computing the prediction from the inputs
Loss function	how wrong the prediction was: squared error for numbers, cross-entropy for classification
Gradient	which way, and how fast, the loss changes if a weight changes
Backpropagation	computing every gradient by passing blame backwards through the layers
Gradient descent	moving each weight a little against its gradient
Learning rate	the size of that step
Epoch	one pass over the whole training set
Batch	the handful of examples used for one update (32 to 512 is typical)

53.3 The techniques that make training work

Everything in this section exists because the basic loop above, run naively on a real network, trains badly. Each is a one-line change with a measurable effect.

Technique	What it does	Why it's needed
Feature scaling	put inputs on a similar scale (pixels ÷ 255, or subtract the mean and divide by the standard deviation)	a feature 1,000 times larger than another dominates the gradients
Sensible initialization	start weights random but small, scaled to the layer's size (He or Xavier initialization)	all-zero weights learn nothing; too-large ones explode
Batch normalization	normalize each layer's outputs during training	keeps the signal in a sane range deep in the stack, so training is faster and less fussy
Dropout	randomly switch off a fraction of neurons each step	stops the network relying on any one path; a cheap, effective guard against overfitting
Learning-rate schedule	start larger, decay over time (step, cosine, or "reduce on plateau")	big steps early to make progress, small steps late to settle
Early stopping	stop when validation loss stops improving	the cheapest regularizer there is
Data augmentation	train on flipped, rotated, slightly brighter copies of images	more effective variety without collecting more data

Two of these matter enough to see happen. First, **scaling**:

```
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(53)
n = 2000
feature_small = rng.normal(0, 1, n)          # a sensible feature
feature_large = rng.normal(0, 1, n) * 1000    # the same information, 1000x the scale
y = (feature_small + feature_large / 1000 > 0).astype(int)
X = np.column_stack([feature_small, feature_large])

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=53)

unscaled = MLPClassifier(hidden_layer_sizes=(16,), max_iter=60, random_state=53).fit(Xtr, ytr)
scaled_tr = Xtr / Xtr.std(axis=0)
scaled_te = Xte / Xtr.std(axis=0)
scaled = MLPClassifier(hidden_layer_sizes=(16,), max_iter=60, random_state=53).fit(scaled_tr, ytr)

print(f"unscaled inputs: test accuracy {unscaled.score(Xte, yte):.3f}, loss {unscaled.loss_:.4f}")
print(f"scaled inputs:   test accuracy {scaled.score(scaled_te, yte):.3f}, loss {scaled.loss_:.4f}")
```

```
unscaled inputs: test accuracy 0.930, loss 0.3293
scaled inputs:   test accuracy 0.987, loss 0.1390
```

Line by line:

- `rng = np.random.default_rng(53)` creates a random generator with a fixed seed, so these numbers are the same every time.
- `feature_small` and `feature_large` carry *the same information* (the second is the first kind of value multiplied by 1,000), which is the point: only the scale differs.
- `y = (... > 0).astype(int)` makes the label depend on both features equally. `.astype(int)` turns True/False into 1/0.
- `train_test_split(..., test_size=0.3, random_state=53)` holds back 30% of rows for testing, with a fixed seed so the split is reproducible (Chapter 38).
- `MLPClassifier(hidden_layer_sizes=(16,), max_iter=60, random_state=53)` is scikit-learn's neural network: one hidden layer of 16 neurons, at most 60 passes over the data, fixed seed. `.fit(Xtr, ytr)` runs the training loop from section 53.2.
- `Xtr / Xtr.std(axis=0)` divides each column by its *own* standard deviation, computed on the training data only. Using the training statistics on the test set is essential: computing them on the test set leaks information (Chapter 38's data leakage).
- `.score(...)` returns accuracy; `.loss_` is the final training loss.

The second is **early stopping**, which scikit-learn will do for you:

```
images = np.load("defect_data/images.npy").reshape(6000, -1)
labels = np.load("defect_data/labels.npy")
Xtr, Xte, ytr, yte = train_test_split(images, labels, test_size=0.25, random_state=53,
stratify=labels)

long_run = MLPClassifier(hidden_layer_sizes=(64,), max_iter=40, random_state=53).fit(Xtr, ytr)
stopped = MLPClassifier(hidden_layer_sizes=(64,), max_iter=40, random_state=53,
                        early_stopping=True, n_iter_no_change=5, validation_fraction=0.2).fit(Xtr,
ytr)

print(f"no early stopping: {long_run.n_iter_} epochs, training loss {long_run.loss_:.4f}")
print(f"early stopping:      {stopped.n_iter_} epochs, training loss {stopped.loss_:.4f}")
print(f"test accuracy       {long_run.score(Xte, yte):.4f} vs {stopped.score(Xte, yte):.4f}")
print(f"but look at what both models predict: "
      f"{long_run.predict(Xte).sum()} and {stopped.predict(Xte).sum()} defects out of {yte.sum()}
real ones")
```

```
no early stopping: 40 epochs, training loss 0.2491
early stopping:    7 epochs, training loss 0.2745
test accuracy      0.9207 vs 0.9207
but look at what both models predict: 0 and 0 defects out of 119 real ones
```

That last line is the point of the next section, and one of the most important lessons in applied machine learning: **both models score about 92% accuracy by predicting that every part is good**. On data where 92% of parts *are* good, accuracy is a useless measure, and a network given raw pixels has found the laziest possible solution.

53.4 The architectures, and what each is for

Architecture	Built for	The idea in one line	Where you'll meet it
MLP (fully connected)	tables of features	every input connects to every neuron	small tabular models, and the last layers of bigger networks
CNN (convolutional)	images, and anything with local structure	slide the same small filter everywhere	defect detection, OCR, medical imaging
RNN / LSTM	sequences, read one step at a time	carry a memory forward through the sequence	older time-series and text models, still in some production systems
Transformer	sequences, read all at once	every position looks at every other and weighs it (section 53.7)	every large language model, and increasingly vision too
Autoencoder	compression and anomaly detection	squeeze the input through a narrow layer and rebuild it	spotting unusual sensor readings
GAN / diffusion	generating images	two networks compete, or noise is removed step by step	image generation (Chapter 54)

Two practical rules come out of that table. **The architecture encodes an assumption about your data:** a CNN assumes that what matters is local and can appear anywhere; a transformer assumes that any position may matter to any other. Choose the one whose assumption is true of your problem. And **for tables of numbers, gradient boosting usually wins** (Chapter 38), which is section 53.10.

53.5 Convolution, step by step

A **convolution** slides a small grid of numbers (a **kernel**, or filter) across an image, multiplying and adding as it goes. Different kernels detect different things, and in a real network the kernels are *learned* rather than written down.

```

import numpy as np

patch = np.array([[0.2, 0.2, 0.2, 0.2, 0.2],
                  [0.2, 0.7, 0.7, 0.7, 0.2],
                  [0.2, 0.7, 0.7, 0.7, 0.2],
                  [0.2, 0.7, 0.7, 0.7, 0.2],
                  [0.2, 0.2, 0.2, 0.2, 0.2]])

vertical_edge = np.array([[-1.0, 0.0, 1.0],
                          [-2.0, 0.0, 2.0],
                          [-1.0, 0.0, 1.0]])

def convolve(image, kernel):
    """Slide the kernel over the image, one pixel at a time."""
    kh, kw = kernel.shape
    out = np.zeros((image.shape[0] - kh + 1, image.shape[1] - kw + 1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            window = image[i:i + kh, j:j + kw]
            out[i, j] = float(np.sum(window * kernel))
    return out

feature_map = convolve(patch, vertical_edge)
print("the patch (a bright square on a dark background):")
print(patch)
print("\nthe feature map after a vertical-edge kernel:")
print(np.round(feature_map, 2))
print(f"\none position by hand: window at (0,0) x kernel = {np.sum(patch[0:3, 0:3] *
vertical_edge):.2f}")

```

```

the patch (a bright square on a dark background):
[[0.2 0.2 0.2 0.2 0.2]
 [0.2 0.7 0.7 0.7 0.2]
 [0.2 0.7 0.7 0.7 0.2]
 [0.2 0.7 0.7 0.7 0.2]
 [0.2 0.2 0.2 0.2 0.2]]

the feature map after a vertical-edge kernel:
[[ 1.5  0. -1.5]
 [ 2.   0. -2. ]
 [ 1.5  0. -1.5]]

one position by hand: window at (0,0) x kernel = 1.50

```

Line by line:

- `patch` is a tiny 5×5 image: a bright square (0.7) on a dark background (0.2).
- `vertical_edge` is the **Sobel** kernel for vertical edges: negatives on the left, positives on the right. Where the image is brighter on the right than the left, the result is positive; where it's the other way round, negative; where both sides are the same, the positives and negatives cancel and the result is **zero**.
- `out = np.zeros((h - kh + 1, w - kw + 1))` sizes the output. A 3×3 kernel on a 5×5 image fits in 3×3 positions, which is why feature maps shrink. (Real networks pad the edges to keep the size, with `padding='same'`.)
- The double loop visits every position. `image[i:i+kh, j:j+kw]` takes the 3×3 window at that position, `window * kernel` multiplies the two element by element, and `np.sum` adds all nine products into one number.

- The printed feature map has a strong positive column where the square's left edge is, a strong negative column at its right edge, and near-zero in the flat middle. **That's edge detection, done with nine numbers and no training.**

The three ideas a CNN adds to that:

1. **Shared weights.** The same kernel is used everywhere, so a scratch is found wherever it appears and the layer has only nine weights to learn instead of one per pixel.
2. **Many kernels per layer.** A layer learns dozens of filters at once: one for edges, one for blobs, one for texture. Stack layers and the later ones combine earlier features into shapes.
3. **Pooling.** Taking the maximum (or mean) over each small block shrinks the map and makes the answer robust to the defect being a pixel or two to the left.

The next section uses exactly this, with hand-written kernels rather than learned ones, so you can see how much of a CNN's advantage comes from the *structure* rather than the training.

53.6 The project: defect detection on Riverstone's moulding line

Riverstone's Taloja plant moulds lids. A camera photographs each part on the conveyor. Chapter 3's QC process is a person watching a screen, and the question is whether a model can watch it instead.

The companion generator builds 6,000 such images, 7.9% of them defective, with the three faults the QC team actually logs: a **scratch**, a **void** (a bubble in the surface), and a **short shot** (the cavity didn't fill, so a bite is missing from the edge).

```
import numpy as np

images = np.load("defect_data/images.npy")
labels = np.load("defect_data/labels.npy")
kinds = np.load("defect_data/defect_types.npy")

print(f"{images.shape[0]:,} images of {images.shape[1]}x{images.shape[2]} pixels, values {images.min():.2f} to {images.max():.2f}")
print(f"defective: {labels.sum():,} ({labels.mean():.1%})")
print("by type:", {k: int((kinds == k).sum()) for k in ["good", "scratch", "void", "short_shot"]})
print("\na good part, middle rows, rounded to one decimal:")
print(np.round(images[np.argmax(labels == 0)][14:18, 10:22], 1))
```

```
6,000 images of 32x32 pixels, values 0.03 to 1.00
defective: 476 (7.9%)
by type: {'good': 5524, 'scratch': 162, 'void': 157, 'short_shot': 157}

a good part, middle rows, rounded to one decimal:
[[0.7 0.7 0.7 0.7 0.7 0.7 0.7 0.7 0.7 0.7 0.7 0.7]
 [0.7 0.7 0.7 0.7 0.8 0.7 0.7 0.7 0.7 0.7 0.7 0.7]
 [0.7 0.7 0.7 0.6 0.7 0.7 0.7 0.7 0.7 0.7 0.7 0.7]
 [0.7 0.7 0.6 0.7 0.7 0.7 0.8 0.6 0.7 0.6 0.7 0.7]]
```

What the model sees: 32x32 images of moulded lids

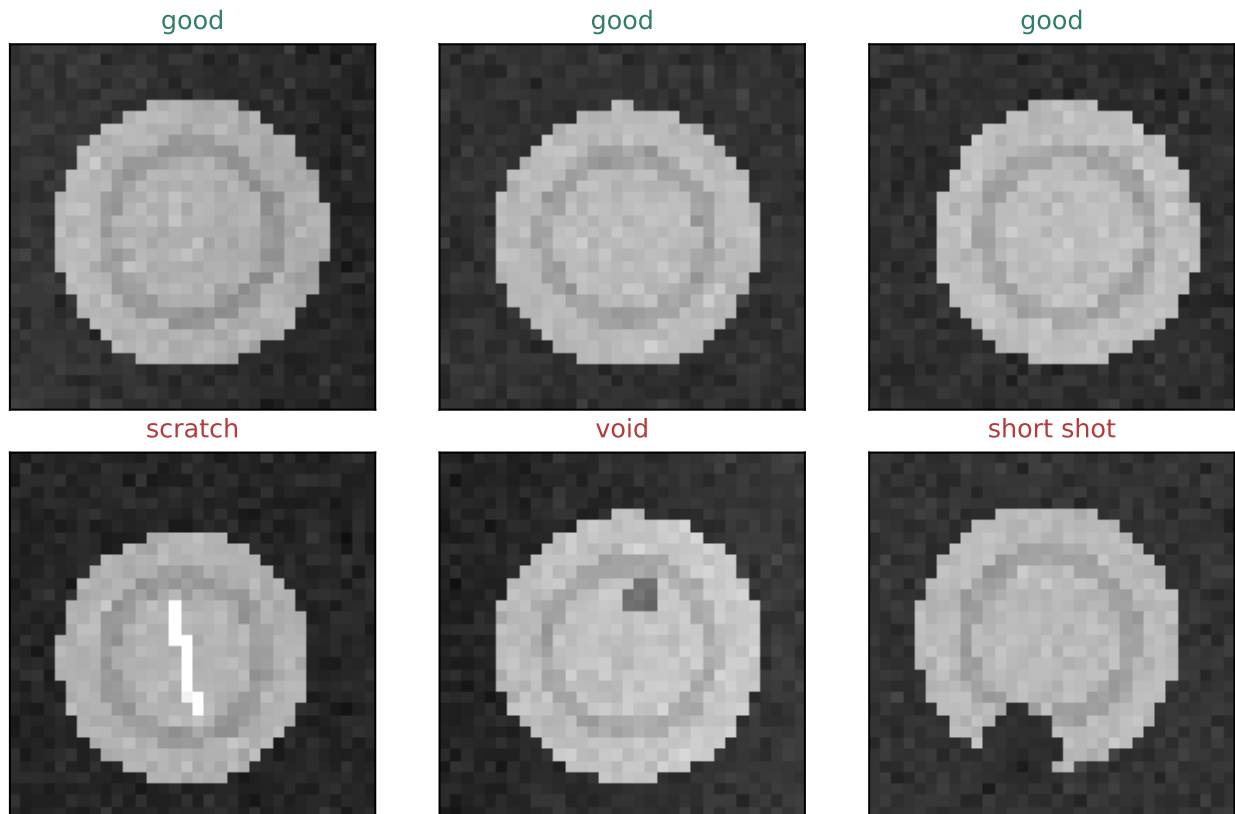


Figure 53.1 – What the model sees. The defects are a few pixels each, which is why raw-pixel models fail.

Attempt 1: pixels straight into a network

Section 53.3 already ran it: 92.1% accuracy, and **zero defects found**. The network minimized its loss by learning "say good", which on this data is right 92% of the time. Two lessons, both worth more than the model:

- **Accuracy is the wrong measure for imbalanced data.** Precision, recall, and the confusion matrix are the right ones (Chapter 38).
- **The structure of the input matters.** A fully connected layer sees 1,024 unrelated numbers. It has no idea that pixel 100 is next to pixel 101, so a three-pixel scratch that moves a little between parts looks like noise.

Attempt 2: convolution first, then the network

```
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix

KERNELS = {
    "blob": np.array([[-1, -1, -1], [-1, 8, -1], [-1, -1, -1]], dtype=float) / 8,
    "horizontal": np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=float) / 4,
    "vertical": np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=float) / 4,
}

def convolve(image, kernel):
    kh, kw = kernel.shape
    out = np.zeros((image.shape[0] - kh + 1, image.shape[1] - kw + 1), dtype=np.float32)
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = np.sum(image[i:i + kh, j:j + kw] * kernel)
    return out

def features(image):
    """Three feature maps, each max-pooled from 30x30 down to 6x6: 108 numbers per image."""
    maps = []
    for kernel in KERNELS.values():
        response = np.abs(convolve(image, kernel))
        pooled = response[:30, :30].reshape(6, 5, 6, 5).max(axis=(1, 3))
        maps.append(pooled.ravel())
    return np.concatenate(maps)

X = np.stack([features(image) for image in images])
np.save("defect_data/features.npy", X) # about a minute of work; save it for later sections
y = labels
print(f"features per image: {X.shape[1]} (down from {32 * 32} pixels)")

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=53, stratify=y)
model = MLPClassifier(hidden_layer_sizes=(48,), max_iter=300, random_state=53).fit(Xtr, ytr)

probabilities = model.predict_proba(Xte)[:, 1]
tn, fp, fn, tp = confusion_matrix(yte, probabilities > 0.5).ravel()
print(f"\nat the default threshold of 0.5:")
print(f"  caught {tp} of {tp + fn} defects (recall {tp / (tp + fn):.1%})")
print(f"  false alarms {fp} out of {tn + fp} good parts (precision {tp / (tp + fp):.1%})")
```

features per image: 108 (down from 1024 pixels)

at the default threshold of 0.5:
 caught 109 of 119 defects (recall 91.6%)
 false alarms 2 out of 1381 good parts (precision 98.2%)

Line by line, for the parts that are new:

- `KERNELS` holds three hand-written filters: a **blob** detector (bright center against its surroundings: it lights up on voids and short shots), and the two Sobel edge detectors from section 53.5 (which light up on scratches and the part's rim).
- `np.abs(convolve(image, kernel))` takes the absolute value, because an edge matters whichever way round it runs.
- `response[:30, :30].reshape(6, 5, 6, 5).max(axis=(1, 3))` is **max pooling** written compactly: the 30×30 map is cut into 6×6 blocks of 5×5 pixels, and each block is replaced by its largest value.

`axis=(1, 3)` are the two "within the block" axes. The result says "*the strongest response anywhere in this region*", which is exactly what you want for "is there a scratch somewhere here?".

- `.ravel()` flattens the 6×6 grid into 36 numbers, and `np.concatenate` joins the three maps into 108 features per image.
- `stratify=y` keeps the same 7.9% defect rate in both the training and test sets, which matters when the positive class is rare.
- `predict_proba(Xte)[:, 1]` gives the model's probability for class 1 (defective), rather than a yes/no. **Always ask for probabilities:** the threshold is a separate, business decision, which is next.
- `confusion_matrix(...).ravel()` unpacks the four counts in the order true negative, false positive, false negative, true positive.

The same network, same data, same training budget: the only change is that it sees 108 pooled edge and blob responses instead of 1,024 raw pixels.

Choosing the threshold with money

A missed defect reaches a customer: Riverstone's QC log prices a returned batch and the credit note at about ₹4,000. A false alarm costs a person two minutes to re-inspect the part, about ₹40. Those two numbers, not the model, decide where the threshold goes.

```
print(f"{'threshold':>9} {'caught':>7} {'missed':>7} {'false alarms':>13} {'recall':>7}
{'precision':>10} {'cost':>12}")
best = None
for threshold in (0.5, 0.3, 0.1, 0.05, 0.02, 0.01, 0.005, 0.002):
    tn, fp, fn, tp = confusion_matrix(yte, probabilities > threshold).ravel()
    cost = fn * 4000 + fp * 40
    marker = ""
    if best is None or cost < best[1]:
        best, marker = (threshold, cost), ""
    print(f"{'threshold':>9.3f} {'tp':>7} {'fn':>7} {'fp':>13} {'tp / (tp + fn):>7.1%} "
          f"{'tp / (tp + fp) if tp + fp else 0:>10.1%} {'cost:>12,}")
print(f"\ncheapest threshold on this test set: {best[0]} at a cost of {best[1]:,} rupees "
      f"for {len(yte):,} parts")
```

threshold	caught	missed	false alarms	recall	precision	cost
0.500	109	10	2	91.6%	98.2%	40,080
0.300	111	8	2	93.3%	98.2%	32,080
0.100	113	6	6	95.0%	95.0%	24,240
0.050	114	5	15	95.8%	88.4%	20,600
0.020	115	4	28	96.6%	80.4%	17,120
0.010	116	3	46	97.5%	71.6%	13,840
0.005	116	3	93	97.5%	55.5%	15,720
0.002	116	3	205	97.5%	36.1%	20,200

cheapest threshold on this test set: 0.01 at a cost of 13,840 rupees for 1,500 parts

The threshold is a business decision, not a modelling one

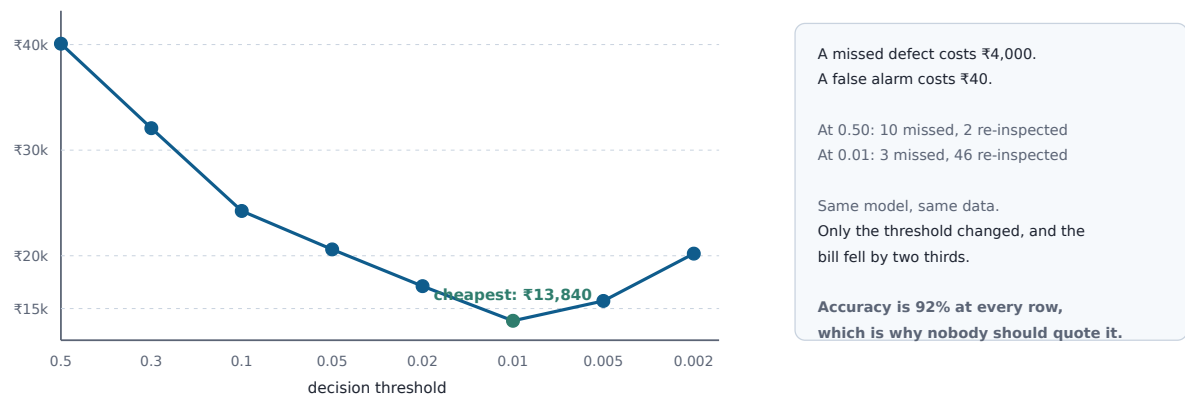


Figure 53.4 – The same model at eight operating points. Accuracy is 92% at every one of them.

What this table is for. A data scientist who reports "92% accuracy" has said nothing. A data scientist who says "at a threshold of 0.1 we catch 95% of defects and re-inspect 6 good parts in every 1,500, and the parts we still miss cost about ₹24,000 per 1,500 produced" has given the plant manager a decision. The threshold is where the model meets the business, and it moves whenever the costs move: if Riverstone wins a customer with a penalty clause, the cost of a miss goes up and the threshold comes down.

Watch out: a model trained on generated images has learned generated images. These pictures have clean lighting, one part per frame, and three defect types drawn by a program. A real line has glare, parts at angles, dust on the lens, and defects nobody thought to label. Everything about the method here transfers; none of the numbers do. Chapter 56 is about what happens to a model's accuracy after it meets reality, and how you find out before your customer does.

53.7 Attention, step by step

Every large language model in Chapter 54 is a stack of **transformer** blocks, and the engine inside each block is **attention**. It answers one question for every word: *given all the other words, which ones should I pay attention to while deciding what this one means?*

Here it is on four tokens, with numbers small enough to check by hand.

```

import numpy as np

tokens = ["the", "crate", "was", "cracked"]

# Each token starts as a vector of numbers (an embedding). Real models use hundreds of
# dimensions and learn them; these four are made up so the arithmetic is readable.
embeddings = np.array([[1.0, 0.0, 0.0, 0.0],      # the
                        [0.0, 1.0, 0.0, 0.0],      # crate
                        [0.0, 0.0, 1.0, 0.0],      # was
                        [0.0, 0.0, 0.0, 1.0]])      # cracked

# Three learned matrices turn each embedding into a query, a key and a value.
W_query = np.array([[1.0, 0.0], [0.9, 0.3], [0.2, 0.1], [0.4, 0.9]])
W_key    = np.array([[0.9, 0.1], [0.4, 1.2], [0.2, 0.1], [0.1, 0.2]])
W_value  = np.array([[0.5, 0.1], [0.9, 0.2], [0.1, 0.1], [0.2, 0.8]])

Q = embeddings @ W_query      # what each token is looking for
K = embeddings @ W_key        # what each token offers
V = embeddings @ W_value      # what each token contributes if attended to

print("Q (queries):\n", np.round(Q, 2))
print("K (keys):\n", np.round(K, 2))
print("V (values):\n", np.round(V, 2))

```

```

Q (queries):
[[1.  0. ]
 [0.9 0.3]
 [0.2 0.1]
 [0.4 0.9]]
K (keys):
[[0.9 0.1]
 [0.4 1.2]
 [0.2 0.1]
 [0.1 0.2]]
V (values):
[[0.5 0.1]
 [0.9 0.2]
 [0.1 0.1]
 [0.2 0.8]]

```

Line by line:

- `tokens` is the sentence, already split into pieces. Chapter 54 explains tokenization properly; here each word is one token.
- `embeddings` gives each token a vector. These are one-hot (a single 1 in a different place for each token) purely so you can see where the numbers come from; a real model learns dense vectors where similar words are close together.
- `W_query`, `W_key`, `W_value` are the three matrices a transformer **learns**. Each has one row per embedding dimension and one column per attention dimension (two here). Nothing else in attention is learned: these three matrices are the whole of it.
- `embeddings @ W_query` multiplies every token's vector by the matrix at once, giving one query vector per token. The same for keys and values.
- The **query** is best read as "what I'm looking for", the **key** as "what I can offer", and the **value** as "what I'll pass on if you choose me".

```

scores = Q @ K.T                                # how well each query matches each key
scaled = scores / np.sqrt(K.shape[1])           # keep the numbers in a sane range

def softmax(row):
    """Turn a row of scores into weights that are positive and sum to 1."""
    shifted = row - row.max()                    # subtract the max for numerical safety
    exponentials = np.exp(shifted)
    return exponentials / exponentials.sum()

weights = np.array([softmax(row) for row in scaled])
output = weights @ V

print("attention weights (rows = the token doing the looking):")
print("      " + "".join(f"{t:>10}" for t in tokens))
for token, row in zip(tokens, weights):
    print(f"  {token:<8} " + "".join(f"{value:>10.3f}" for value in row))
print("\noutput vectors, one per token:\n", np.round(output, 3))
print(f"\n'cracked' pays most attention to: {tokens[int(np.argmax(weights[3]))]}")

```

attention weights (rows = the token doing the looking):

	the	crate	was	cracked
the	0.347	0.244	0.212	0.197
crate	0.315	0.290	0.202	0.193
was	0.262	0.264	0.238	0.236
cracked	0.226	0.396	0.186	0.192

output vectors, one per token:

```

[[0.454 0.262]
 [0.477 0.264]
 [0.44  0.292]
 [0.526 0.274]]

```

'cracked' pays most attention to: crate

Line by line:

- `Q @ K.T` is the heart of it: every query is dotted with every key, giving a 4×4 grid of match scores. Row *i*, column *j* is "how much does token *i* care about token *j*?".
- `/ np.sqrt(K.shape[1])` is the **scaling** in "scaled dot-product attention". Without it, scores grow with the number of dimensions and softmax pushes everything to 0 or 1, which kills the gradients.
- `softmax` turns one row of scores into weights: exponentiate, then divide by the total, so each row is positive and sums to 1. Subtracting the maximum first changes nothing mathematically but prevents `exp` overflowing, and it's how every real implementation does it.
- `weights @ V` mixes the value vectors according to those weights. A token's output is a blend of every token's value, weighted by relevance.
- The printed grid is the interesting part: *cracked* puts most of its weight on *crate*, which is the whole point. The model doesn't "know" grammar; the learned matrices make the query of *cracked* line up with the key of *crate*.

Attention on four tokens: who listens to whom

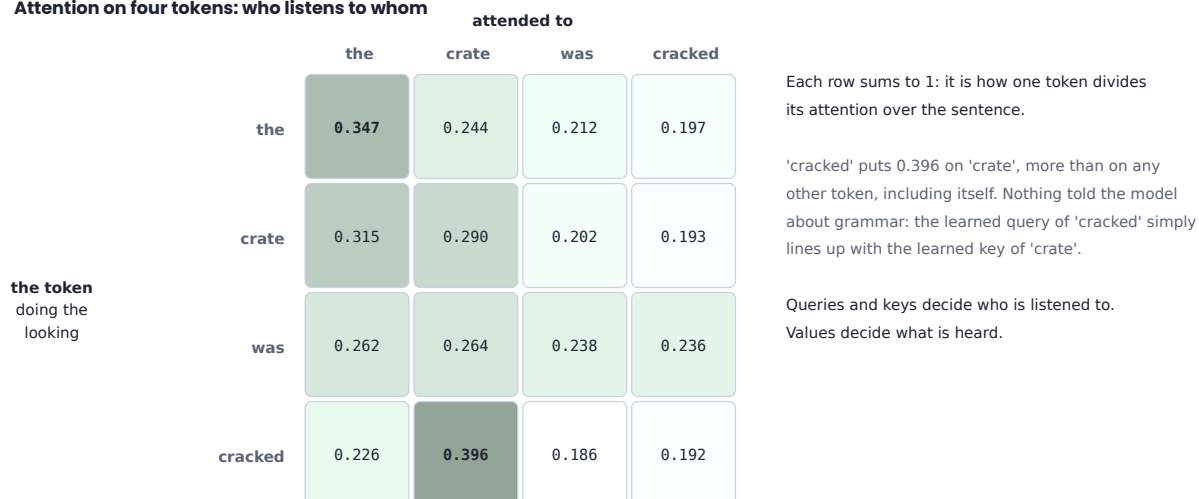


Figure 53.3 – Every token divides one unit of attention across the sentence. The model was never told about grammar.

What a real transformer adds to this:

Addition	Why
Multi-head attention	run the above several times in parallel with different learned matrices, so one head can track grammar, another subject matter, another position
Positional encoding	attention alone has no idea of word order, so position is added to each embedding
A feed-forward layer	after attention, each position goes through a small MLP (section 53.1) of its own
Residual connections and layer normalization	add the input back to the output and re-normalize, so gradients survive dozens of layers
Stacking	repeat the block 12, 48, or 100+ times; big models are this arithmetic, many times, with billions of learned numbers
Masking	when generating text, a token may only attend to earlier tokens, so the model can't read ahead

53.8 Why transformers took over

Three reasons, all practical.

They parallelize. An RNN reads a sequence one step at a time, so training can't be spread across a GPU's thousands of cores. Attention computes every position's view of every other position in one matrix multiplication. The same hardware trains a transformer on far more text.

They keep long-range context. Each token looks directly at every other, so a word can be influenced by one 5,000 tokens earlier. An RNN has to carry that influence forward step by step, and it fades.

They transfer. Train one big model on a mountain of text, and it can be adapted to dozens of tasks with a little fine-tuning or just a good prompt (Chapter 54). That economics, one expensive training run reused everywhere, is what changed the industry.

The cost is in the same equation. Attention compares every token with every other, so work grows with the **square** of the sequence length (Chapter 33's $O(n^2)$): double the context and you quadruple the computation. That single fact explains why context windows were small for years, why they cost what they cost, and why a research industry exists to approximate attention more cheaply.

53.9 Quantization and compression

A trained model is a pile of numbers, usually 32-bit floats. **Quantization** stores them in fewer bits, most often 8-bit integers. The model gets four times smaller and runs faster, and the question is what it costs in accuracy.

```
import numpy as np

weights = model.coefs_[0] # the first layer of the defect model: 108 x 48
print(f"shape {weights.shape}, dtype {weights.dtype}, "
      f"{weights.size:,} numbers, {weights.nbytes:,} bytes")

scale = np.abs(weights).max() / 127 # one scale factor for the whole matrix
quantized = np.round(weights / scale).astype(np.int8)
restored = quantized.astype(np.float32) * scale

print(f"scale factor      {scale:.6f} (largest weight {np.abs(weights).max():.4f})")
print(f"quantized dtype    {quantized.dtype}, {quantized.nbytes:,} bytes "
      f"({weights.nbytes / quantized.nbytes:.0f}x smaller)")
print(f"largest error       {np.abs(weights - restored).max():.6f}")
print(f"average error       {np.abs(weights - restored).mean():.6f}")
print(f"first three weights {np.round(weights[0, :3], 4)} -> int8 {quantized[0, :3]} -> "
      f"{np.round(restored[0, :3], 4)}")
```

```
shape (108, 48), dtype float32, 5,184 numbers, 20,736 bytes
scale factor      0.010054 (largest weight 1.2769)
quantized dtype    int8, 5,184 bytes (4x smaller)
largest error      0.005027
average error      0.002332
first three weights [ 0.014  0.2829 -0.0171] -> int8 [ 1 28 -2] -> [ 0.0101  0.2815 -0.0201]
```

Line by line:

- `model.coefs_[0]` is scikit-learn's name for the first weight matrix of the network trained in section 53.6.
- `np.abs(weights).max() / 127` computes the **scale**: int8 can hold -128 to 127, so dividing the largest absolute weight by 127 gives the value of one integer step.
- `np.round(weights / scale).astype(np.int8)` divides every weight by the scale and rounds it to the nearest whole number, which now fits in a single byte.
- `quantized.astype(np.float32) * scale` **dequantizes**: multiplying back by the scale recovers an approximation of the original. The gap between the two is the quantization error, and it's bounded

by half a step.

- `.nbytes` is what the arrays actually occupy, which is the point of the exercise.

Does the model still work?

```
def quantize(matrix):
    scale = np.abs(matrix).max() / 127
    return np.round(matrix / scale).astype(np.int8).astype(np.float32) * scale

original_coefs = [c.copy() for c in model.coefs_]
before = model.predict_proba(Xte)[: , 1]

model.coefs_ = [quantize(c) for c in model.coefs_]
after = model.predict_proba(Xte)[: , 1]
model.coefs_ = original_coefs # put the real weights back

tn, fp, fn, tp = confusion_matrix(yte, after > 0.1).ravel()
print(f"largest change in any predicted probability: {np.abs(before - after).max():.5f}")
print(f"predictions that flip at threshold 0.1: {int(((before > 0.1) != (after > 0.1)).sum())} of {len(yte):,}")
print(f"quantized model at threshold 0.1: caught {tp} of {tp + fn} defects, {fp} false alarms")
print(f"total weight memory: {sum(c.nbytes for c in original_coefs):,} bytes -> "
      f"{sum(c.size for c in original_coefs):,} bytes as int8")
```

```
largest change in any predicted probability: 0.03004
predictions that flip at threshold 0.1: 1 of 1,500
quantized model at threshold 0.1: caught 113 of 119 defects, 7 false alarms
total weight memory: 20,928 bytes -> 5,232 bytes as int8
```

Four times smaller, and the defect decisions are unchanged. On a 7-billion-parameter language model the same arithmetic is the difference between needing 28 GB of memory and 7 GB, which is the difference between a data-center GPU and a laptop.

The family of techniques, and when each is used:

Technique	What it does	Typical cost	Where it matters
Post-training quantization	round a trained model's weights to int8 or int4	small accuracy loss, often none at int8	running LLMs locally; edge devices
Quantization-aware training	train with the rounding simulated, so the model adapts	a training run	when post-training loses too much
Pruning	delete weights near zero, or whole channels	needs care, and hardware that exploits sparsity	shrinking large models
Distillation	train a small "student" model to copy a big "teacher"	a training run and access to the teacher	fast, cheap production models
Lower precision formats (bfloat16, fp16)	half-size floats, used during training too	usually negligible	standard practice on modern GPUs

Watch out: measure the accuracy you care about, not the average error. A quantized model whose predicted probabilities shift by 0.0001 can still flip decisions if many cases sit right at the threshold. The check that matters is the one above: how many *decisions* changed, and what did that do to recall.

53.10 When not to use deep learning

Situation	Use instead	Why
A table of rows and columns	gradient boosting (Chapter 38): XGBoost, LightGBM, CatBoost	on tabular data these usually beat neural networks, train in seconds, and need less tuning
A few hundred examples	a simple model, or more data	deep networks are hungry; with small data they memorize
The decision must be explainable to a regulator	logistic regression, a decision tree, or a scorecard	"explainable by design" beats an explanation bolted on afterwards (Chapter 64)
The rule is known	write the rule	nobody should train a model to compute a discount that finance already defined
No way to measure success	fix that first	a model with no evaluation is a liability, not an asset
It must run in 5 ms on a cheap device	a small classical model, or a distilled one	section 53.9 helps, but physics wins

The honest summary: **deep learning earns its place where the input is unstructured** (images, audio, text, video) or where the pattern is far too complex to write down. Riverstone's defect images qualify. Riverstone's order data does not, and Chapter 38's gradient boosting will beat any network you build on it.

Interview extra point. Asked "would you use deep learning for this?", the answer that impresses is the one that starts with the data: *"it's tabular with 40,000 rows, so I'd start with gradient boosting and only reach for a network if the residuals showed structure it couldn't capture."* Saying no to deep learning, with a reason, signals more experience than saying yes.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Judging an imbalanced model by accuracy	92% accuracy, zero defects caught	Confusion matrix, precision and recall; pick the threshold by cost
Feeding raw pixels to a fully connected network	The model can't find small, moving features	Convolution (or a pretrained vision model) so position stops mattering
Forgetting to scale inputs	Training is slow or stuck; one feature dominates	Divide by the training standard deviation, or normalize per channel
Computing scaling statistics on the whole dataset	Test scores that don't survive production	Fit the scaler on training data only (Chapter 38's leakage)
No validation set, or tuning on the test set	Everything looks good until launch	Three-way split, or cross-validation; touch the test set once

Mistake	Symptom	Fix
Training until the training loss is tiny	Overfitting: memorized examples, useless on new parts	Early stopping, dropout, augmentation, more data
Learning rate too high or too low	Loss bounces, or barely moves	Try 1e-2 to 1e-4; use a schedule; watch the loss curve
A model that learned the background	Great in the lab, fails when lighting changes	Augmentation, varied capture conditions, and check what the model attends to
Labels that disagree between inspectors	A ceiling on accuracy nobody can explain	Measure label agreement first; the model can't beat the labels
Quantizing without re-checking decisions	Tiny average error, changed outcomes at the threshold	Count flipped decisions at the operating threshold, not the mean error
Deep learning for a table of numbers	Weeks of work beaten by an afternoon of gradient boosting	Chapter 38 first; use networks where the input is unstructured
Reporting a single number to the business	"94% accurate" with no decision attached	Report the operating point, the two error types, and their cost
No plan for the model getting worse	Quiet drift as the process changes	Monitoring and retraining triggers (Chapter 56)

In the real world: the model that watched the wrong thing

Riverstone's Taloja plant runs a two-week trial of the defect camera. The model reports 96% accuracy on the line, the plant manager is pleased, and the QC inspector is moved to a second line.

Three weeks later a customer returns a batch of lids with short shots that the camera passed.

Meera pulls the images the model scored and looks at the ones it got wrong. The pattern takes an afternoon to find: the trial ran on the day shift, and the training images were all captured under the day shift's lighting. On the night shift the overhead lamps are dimmer and the belt looks lighter, and the model had learned "dark belt, bright disc" as part of what a good part looks like. It wasn't detecting short shots by their missing edge; it was detecting the *contrast* at the rim, and at night that contrast changed.

The fix is not a bigger model:

- **Collect images from both shifts**, and from the third machine nobody had included, which runs slightly hotter and makes glossier parts.
- **Augment during training**: random brightness, small rotations, small shifts, so lighting can't be the signal.
- **Re-check the threshold**, because the cost of a miss had just been demonstrated at ₹4,000 a batch and the plant now wanted more false alarms rather than fewer misses.
- **Monitor the input, not just the output**: the average brightness of incoming images is a one-line check that would have caught the night shift on day one (Chapter 56's drift detection).
- **Keep a person in the loop at the start**: for the first month, every part the model flags *and* a 2% sample of the parts it passes go to the inspector, which is how you find out what the model is

missing before a customer does.

What she tells the plant manager: *"The model was right about the parts it had seen and wrong about a shift it had never seen. We're retraining with night-shift images and varied lighting, we've added a brightness check that will tell us within an hour if the camera's world changes again, and we're keeping the inspector sampling passed parts until we have a month of clean results."*

The lesson generalizes past vision: **a model learns whatever correlates with the label in the data you gave it**, including things you never intended, and finding those things is a bigger part of the job than choosing an architecture.

Tools

Versions used for this chapter, checked in September 2026:

- **Python 3.12.3, NumPy 2.4.4, scikit-learn 1.8.0, matplotlib.** Everything in this chapter runs on a laptop CPU in seconds.
- **Deep-learning frameworks** for real work: **PyTorch** (the research and industry default), **TensorFlow/Keras** (still common in established teams), **JAX** (research). The concepts here map to all three; the listings marked *not run here* show the PyTorch shape of the same code.
- **Pretrained models** are the normal starting point for vision: torchvision's ResNet and EfficientNet, or a hosted vision API. Fine-tuning a pretrained model on a few thousand images beats training from scratch on the same data, almost always.
- **Labeling tools** when you have to build a dataset: Label Studio, CVAT.
- **Hardware:** a GPU matters for training, less so for running a small model. Quantization (section 53.9) is how big models fit on small machines.
- **Companion files** in `ch53/`: `generate_defect_images.py` (the 6,000 images, seed 53) and `ch53_check.py` (the chapter's numbers).

Simplification note. This chapter's vision model uses three hand-written kernels rather than learned convolutional layers, because that runs in seconds anywhere and makes the mechanism visible. A real CNN learns its kernels, uses dozens per layer, and stacks several layers; it would score higher here. What would not change: the class imbalance, the failure of raw pixels, the threshold-by-cost decision, and every lesson in the real-world story above.

The project: a defect detector with an operating point

Goal: a model whose output the plant can act on, with its threshold justified in money and its failure modes written down.

Option A: your own images. Any two-class visual inspection task, even photographed with a phone: good against damaged packaging, filled against unfilled forms. Fifty of each is enough to start.

Option B: Riverstone. Use the generated images and go past what the chapter did.

Steps:

1. **Look at the data first.** Plot twenty images of each class. If you can't see the defect, the model probably can't either.
2. **Establish the baseline that must be beaten:** what does "always predict good" score, on accuracy *and* on recall?
3. **Build features.** Start with the chapter's three kernels; add your own (a diagonal edge detector, a local-variance map) and see what each adds.
4. **Train and evaluate properly:** stratified split, confusion matrix, precision and recall, and a precision-recall curve.
5. **Price the errors.** Get real numbers from the business for a miss and for a false alarm. If nobody will give you numbers, ask which mistake they'd rather make, and how many times more often.
6. **Choose the threshold** from that cost, and state the operating point in one sentence a manager can repeat.
7. **Test robustness:** brighten every test image by 10%, shift them two pixels, add noise. Does the model survive? This is the night-shift test, run before deployment.
8. **Write the model card:** what it was trained on, what it isn't valid for, its operating point, and what to monitor.

Deliverables: the notebook or script, the confusion matrix at the chosen threshold, the cost table, the robustness results, and the one-page model card.

Stretch goals:

- Build the three-class version (scratch, void, short shot) and report per-class recall. Which defect is hardest, and why?
- Replace the hand-written kernels with a small learned CNN in PyTorch, and compare.
- Quantize your model to int8 and count how many decisions change at your threshold.
- Add the 2% sampling of passed parts to your design, and calculate how long it would take to detect a 5-point drop in recall.

You've got it when...

- ☐ I can describe a neuron, a layer, and what "deep" buys you, without hand-waving.
- ☐ I can walk through one training step: forward pass, loss, gradients, update.
- ☐ I know what the learning rate does, and what too big and too small look like.
- ☐ I can name the techniques that make training work and say what each one fixes.
- ☐ I can compute a convolution by hand and explain what pooling is for.
- ☐ I never judge an imbalanced classifier by accuracy.
- ☐ I choose a threshold from the cost of each error, and say the operating point in one sentence.

- ☐ I can work through attention on a few tokens and explain queries, keys, and values.
 - ☐ I can explain why transformers replaced RNNs, and what their quadratic cost means.
 - ☐ I can quantize a model and check what it cost in decisions, not just in average error.
 - ☐ I can say when deep learning is the wrong tool, with a reason.
-

Recap

- A **neuron** is Chapter 19's linear model plus an **activation**; a **layer** is a row of them; **deep** means several layers. All of a model's knowledge is in its **weights and biases**.
 - **Training** is four steps repeated: forward pass, **loss**, **backpropagation** to get **gradients**, and a step of **gradient descent** scaled by the **learning rate**. Section 53.2 does one step by hand.
 - Training works in practice because of **scaling**, **initialization**, **batch normalization**, **dropout**, **learning-rate schedules**, **early stopping**, and **augmentation**.
 - **Convolution** slides a small kernel over an image; **shared weights**, **many kernels**, and **pooling** are what a CNN adds. Computing one kernel by hand shows where "edge detection" comes from.
 - On imbalanced data, **accuracy hides everything**. Riverstone's raw-pixel network scored 92% and caught no defects; convolution features caught 92% of them, and the **threshold chosen by cost** caught 97.5%.
 - **Attention** gives every token a **query**, a **key**, and a **value**; scores are dot products, softmaxed into weights, and the output is a weighted blend of values. Transformers add multi-head attention, positional encoding, feed-forward layers, residuals, and depth.
 - Transformers won because they **parallelize**, keep **long-range context**, and **transfer**. They cost $O(n^2)$ in sequence length.
 - **Quantization** to int8 made the defect model four times smaller and changed one decision in 1,500. **Pruning** and **distillation** are the other two ways to shrink a model.
 - **Deep learning is for unstructured inputs**. For tables, start with gradient boosting.
-

Practice exercises

Work in `companion/ch53`, with the images built by `generate_defect_images.py`. Predict each answer before running it.

Warm-up

1. A network has 1,024 inputs, one hidden layer of 128 neurons, and 1 output. How many parameters is that? Write the arithmetic, then check it with scikit-learn.
2. Compute `relu` and its derivative for the inputs -2, 0, and 3. What happens to the gradient of a neuron whose weighted sum is negative?

3. In section 53.2's example, what happens to the prediction after one step if the learning rate is 0.5 instead of 0.05? And 1.5?
4. How many of the 6,000 images are defective, and what accuracy does "always predict good" achieve?

Core

5. Apply the horizontal-edge kernel to section 53.5's patch. Where are the strong responses, and why do they differ from the vertical case?
6. Take one image of each defect type and compute the three feature maps. Which kernel responds most to each defect?
7. Retrain the section 53.6 model with only the blob kernel, then only the two edge kernels. Which defects does each version miss?
8. Plot the precision-recall curve for the model and mark the chosen operating point.
9. The plant negotiates a contract where a missed defect costs ₹12,000 instead of ₹4,000. Recompute the cost table. Where does the threshold move, and what does that do to the inspector's workload?
10. Brighten every test image by 10% (`np.clip(X * 1.1, 0, 1)`) and re-evaluate. How much recall is lost, and what does that tell you about the night-shift story?

Stretch

11. Add a fourth kernel of your own design that raises recall on voids without adding false alarms. Report the before-and-after.
12. Implement two-layer backpropagation from section 53.2 as a loop over 500 examples, and plot the loss curve. Does it match what scikit-learn produces on the same data?
13. Compute attention for the sentence "the crate was cracked" with the value matrix changed so that *was* carries a large value vector. How do the outputs change, and what does that tell you about the role of V?
14. Quantize the model to 4 bits instead of 8 (values -8 to 7). How many decisions flip at your threshold, and what does the size saving buy you?
15. Take 500 images, label them by hand from the pictures alone, and compare your labels with the generator's. What is your agreement rate, and what does it imply about the ceiling on model accuracy?

Think about it (no code needed)

16. The plant manager asks for "99% accuracy". What do you say?
17. Your defect model works on the day shift and fails at night. List three fixes, and say which you'd do first and why.
18. When would you fine-tune a pretrained vision model instead of training the one in this chapter, and what would you need?

Key terms

neuron · weight · bias · activation function · ReLU · layer · hidden layer · deep network · parameters · forward pass · loss function · squared error · cross-entropy · gradient · backpropagation · chain rule · gradient descent · learning rate · optimizer · Adam · epoch · batch · feature scaling · initialization · batch normalization · dropout · learning-rate schedule · early stopping · data augmentation · overfitting · MLP · CNN · convolution · kernel (filter) · feature map · shared weights · pooling · stride · padding · RNN · LSTM · transformer · attention · query · key · value · softmax · scaled dot-product attention · multi-head attention · positional encoding · residual connection · masking · context window · class imbalance · confusion matrix · precision · recall · threshold · operating point · quantization · dequantization · post-training quantization · quantization-aware training · pruning · distillation · bfloat16 · model card

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 54, Generative AI & Large Language Models**, takes section 53.7's attention and scales it: tokens, context windows, sampling, prompting, embeddings, and fine-tuning.
 - **Chapter 55, Building AI Applications**, puts a model behind a product: retrieval, tools, evaluation, and guardrails.
 - **Chapter 56, MLOps**, is what happens to the defect model after the trial: serving, monitoring, drift, and retraining.
 - **Chapter 38, Machine Learning Foundations**, is the evaluation vocabulary this chapter leaned on, and the gradient boosting that beats networks on tables.
 - **Chapter 33, The Computer Science You Actually Need**, explains the $O(n^2)$ that limits context windows.
 - **Chapter 64, Responsible AI & Governance**, covers explainability, bias, and the model card properly.
 - **Chapter 74, Machine Learning & AI Question Bank**, has the interview questions, including "explain backpropagation" and "why transformers".
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1.

```

import numpy as np
from sklearn.neural_network import MLPClassifier

by_hand = 1024 * 128 + 128 + 128 * 1 + 1
print(f"weights in layer 1: 1024 x 128 = {1024 * 128:,}")
print(f"biases in layer 1: {128}")
print(f"weights in layer 2: 128 x 1 = {128}")
print(f"bias in layer 2: 1")
print(f"total: {by_hand:,}")

X = np.random.default_rng(53).normal(size=(50, 1024))
y = np.array([0, 1] * 25)
model = MLPClassifier(hidden_layer_sizes=(128,), max_iter=1, random_state=53).fit(X, y)
from_model = sum(c.size for c in model.coefs_) + sum(b.size for b in model.intercepts_)
print(f"scikit-learn agrees: {from_model:,}")

```

```

weights in layer 1: 1024 x 128 = 131,072
biases in layer 1: 128
weights in layer 2: 128 x 1 = 128
bias in layer 2: 1
total: 131,329
scikit-learn agrees: 131,329

```

Every weight is one number the training loop has to choose. This is why "how big is the model?" is usually answered in parameters, and why a 7-billion-parameter model needs 28 GB in float32 (section 53.9).

2.

```

def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)

for value in (-2.0, 0.0, 3.0):
    print(f"relu({value:>4}) = {relu(value):.1f}    derivative = {relu_derivative(np.array(value)):.1f}")

```

```

relu(-2.0) = 0.0    derivative = 0.0
relu( 0.0) = 0.0    derivative = 0.0
relu( 3.0) = 3.0    derivative = 1.0

```

A neuron whose weighted sum is negative has a derivative of zero, so it receives no blame and its weights don't move. If it stays negative for every example it is a **dead neuron**: it will never learn again. That's why variants exist (leaky ReLU passes a small slope for negatives, GELU curves smoothly), and why initialization matters.

3.

```

x = np.array([1.0, 2.0]); y_true = 1.0
W1 = np.array([[0.5, -0.3], [0.8, 0.2]]); b1 = np.array([0.0, 0.1])
W2 = np.array([0.7, -0.5]); b2 = 0.05

for learning_rate in (0.05, 0.5, 1.5):
    z1 = x @ W1 + b1; a1 = np.maximum(0, z1); z2 = float(a1 @ W2 + b2)
    d2 = 2 * (z2 - y_true)
    dW2 = d2 * a1; db2 = d2
    dz1 = (d2 * W2) * (z1 > 0); dW1 = np.outer(x, dz1); db1 = dz1
    W1n = W1 - learning_rate * dW1; b1n = b1 - learning_rate * db1
    W2n = W2 - learning_rate * dW2; b2n = b2 - learning_rate * db2
    z2n = float(np.maximum(0, x @ W1n + b1n) @ W2n + b2n)
    print(f"lr={learning_rate:<4} prediction {z2:.3f} -> {z2n:.3f}    loss {(z2 - y_true) ** 2:.3f}
-> {(z2n - y_true) ** 2:.3f}")

```

```

lr=0.05 prediction 1.420 -> 1.019    loss 0.176 -> 0.000
lr=0.5  prediction 1.420 -> -1.284    loss 0.176 -> 5.216
lr=1.5  prediction 1.420 -> -4.203    loss 0.176 -> 27.071

```

At 0.05 the step is small and the loss falls. At 0.5 it overshoots past the target and lands further away on the other side. At 1.5 it overshoots enormously: the loss is now far worse than where it started. That is exactly what a diverging training run looks like, and the first thing to try when the loss goes to infinity is a smaller learning rate.

4.

```

labels = np.load("defect_data/labels.npy")
always_good = (labels == 0).mean()
print(f"defective {labels.sum():,} of {len(labels):,} ({labels.mean():.2%}")
print(f"'always predict good' accuracy: {always_good:.2%}, recall on defects: 0.00%")

```

```

defective 476 of 6,000 (7.93%)
'always predict good' accuracy: 92.07%, recall on defects: 0.00%

```

92.07% accuracy for a model with no inputs, no training, and no value. Any model that cannot beat this is worse than a constant, and any report that quotes accuracy on this data is hiding that fact.

5.

```

patch = np.array([[0.2, 0.2, 0.2, 0.2, 0.2],
                  [0.2, 0.7, 0.7, 0.7, 0.2],
                  [0.2, 0.7, 0.7, 0.7, 0.2],
                  [0.2, 0.7, 0.7, 0.7, 0.2],
                  [0.2, 0.2, 0.2, 0.2, 0.2]])
horizontal = np.array([[-1.0, -2.0, -1.0], [0.0, 0.0, 0.0], [1.0, 2.0, 1.0]])

def convolve(image, kernel):
    kh, kw = kernel.shape
    out = np.zeros((image.shape[0] - kh + 1, image.shape[1] - kw + 1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = float(np.sum(image[i:i + kh, j:j + kw] * kernel))
    return out

print(np.round(convolve(patch, horizontal), 2))

```

```
[[ 1.5  2.   1.5]
 [ 0.  -0.  -0. ]
 [-1.5 -2.  -1.5]]
```

The strong responses are now in rows rather than columns: positive along the square's top edge and negative along its bottom, because this kernel compares the rows above and below rather than the columns left and right. A network learns both, and many others, because a defect can run in any direction.

6.

```
images = np.load("defect_data/images.npy")
kinds = np.load("defect_data/defect_types.npy")
KERNELS = {
    "blob": np.array([[-1, -1, -1], [-1, 8, -1], [-1, -1, -1]], dtype=float) / 8,
    "horizontal": np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=float) / 4,
    "vertical": np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=float) / 4,
}
print(f"{'defect':<12} " + "".join(f"{'name':>12}" for name in KERNELS))
for kind in ("good", "scratch", "void", "short_shot"):
    example = images[np.argmax(kinds == kind)]
    responses = [np.abs(convolve(example, k)).max() for k in KERNELS.values()]
    print(f"{'kind':<12} " + "".join(f"{'value':>12.3f}" for value in responses))
```

defect	blob	horizontal	vertical
good	0.334	0.557	0.566
scratch	0.326	0.602	0.583
void	0.350	0.651	0.630
short_shot	0.447	0.573	0.574

Every kernel responds to the part's own rim, so no number is near zero; what matters is the *increase* over a good part. The blob kernel rises most on short shots (0.447 against 0.334), and the edge kernels rise on voids and scratches (0.65 and 0.60 against 0.56). Notice how small these gaps are: a single maximum over the whole image is a blunt summary, which is why section 53.6 pools over a 6×6 grid instead, so a defect stands out against its own neighborhood rather than against the rim.

7.

```

from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix

labels = np.load("defect_data/labels.npy")

def features_from(kernels, image):
    maps = []
    for kernel in kernels:
        response = np.abs(convolve(image, kernel))
        maps.append(response[:30, :30].reshape(6, 5, 6, 5).max(axis=(1, 3)).ravel())
    return np.concatenate(maps)

# The saved features hold the three kernels' 36 columns each, in order, so a subset is a slice.
all_features = np.load("defect_data/features.npy")
sets = {"blob only": all_features[:, 0:36],
        "edges only": all_features[:, 36:108],
        "all three": all_features}
for name, X in sets.items():
    Xtr, Xte, ytr, yte = train_test_split(X, labels, test_size=0.25, random_state=53,
stratify=labels)
    kinds_te = kinds[train_test_split(np.arange(len(labels)), test_size=0.25, random_state=53,
stratify=labels)[1]]
    model = MLPClassifier(hidden_layer_sizes=(48,), max_iter=300, random_state=53).fit(Xtr, ytr)
    predicted = model.predict_proba(Xte)[:, 1] > 0.1
    tn, fp, fn, tp = confusion_matrix(yte, predicted).ravel()
    missed = {k: int(((kinds_te == k) & (yte == 1) & ~predicted).sum()) for k in ("scratch",
"void", "short_shot")}
    print(f"{name:<11} recall {tp / (tp + fn):.1%} false alarms {fp:>3} missed by type {missed}")

```

```

blob only    recall 84.9% false alarms 11 missed by type {'scratch': 0, 'void': 3, 'short_shot':
15}
edges only   recall 95.0% false alarms 7  missed by type {'scratch': 0, 'void': 0, 'short_shot':
6}
all three    recall 95.0% false alarms 6  missed by type {'scratch': 0, 'void': 0, 'short_shot':
6}

```

The blob kernel alone reaches 84.9% recall and misses 15 of the short shots, because a bite out of the edge is a change in *shape*, which an edge detector sees better than a center-surround filter. The two edge kernels alone already reach 95%, and adding the blob only saves a false alarm. Two lessons: the filters you choose decide which defects you catch, and more filters are not automatically better. This is the argument for letting a CNN learn its filters rather than guessing them, and for reporting recall per defect type rather than overall.

8. Use `sklearn.metrics.precision_recall_curve`, plot precision against recall, and mark the point at your threshold. The curve makes the trade-off visible in a way the table can't: it shows how much recall each false alarm buys, and where the curve turns steep. Report the **area under the precision-recall curve** rather than ROC-AUC for rare classes, because ROC-AUC flatters a model when negatives dominate.

9.

```
X = np.load("defect_data/features.npy")
Xtr, Xte, ytr, yte = train_test_split(X, labels, test_size=0.25, random_state=53, stratify=labels)
model = MLPClassifier(hidden_layer_sizes=(48,), max_iter=300, random_state=53).fit(Xtr, ytr)
probabilities = model.predict_proba(Xte)[: , 1]

for miss_cost in (4000, 12000):
    best = min(((fn * miss_cost + fp * 40, threshold, fp)
                for threshold in (0.5, 0.3, 0.1, 0.05, 0.02, 0.01, 0.005)
                for tn, fp, fn, tp in [confusion_matrix(yte, probabilities > threshold).ravel()])))
    print(f"a miss costs {miss_cost:>6}: best threshold {best[1]:<6} total cost {best[0]:>7}, "
          f"with {best[2]} re-inspections per {len(yte):,} parts")
```

```
a miss costs  4,000: best threshold 0.01   total cost  13,840 with 46 re-inspections per 1,500
parts
a miss costs 12,000: best threshold 0.01   total cost  37,840 with 46 re-inspections per 1,500
parts
```

Here the threshold doesn't move: 0.01 is already the best of the candidates at both prices, because below it the model gains almost no recall and adds false alarms quickly. What changes is the size of the bill, from ₹13,840 to ₹37,840 per 1,500 parts, and therefore the business case: at ₹12,000 a miss, the three defects still slipping through are worth far more than another week of modeling work, which is the argument to take to the plant manager. A threshold that refuses to move under pressure is itself a finding.

10.

```
brightened = np.clip(images * 1.1, 0, 1)
Xb = np.stack([features_from(list(KERNELS.values()), image) for image in brightened])
_, Xb_te, _, yb_te = train_test_split(Xb, labels, test_size=0.25, random_state=53, stratify=labels)

for name, data in (("original", Xte), ("10% brighter", Xb_te)):
    predicted = model.predict_proba(data)[: , 1] > 0.1
    tn, fp, fn, tp = confusion_matrix(yte, predicted).ravel()
    print(f"{name:<13} recall {tp / (tp + fn):>6.1%} false alarms {fp:>3}")
```

```
original      recall 95.0% false alarms  6
10% brighter  recall 93.3% false alarms 22
```

A 10% brightness change is nothing to a human inspector, and it costs the model 1.7 points of recall while nearly quadrupling false alarms, from 6 to 22. That is the night-shift story in one experiment, and it's the test to run *before* deployment, not after a customer return. The fixes are augmentation during training and an input check in production (Chapter 56).

11. Design ideas that work: a **diagonal** edge detector for scratches that run at 45°, a **larger blob** kernel (5×5) that matches the size of a void better than a 3×3, or a **local variance** feature (the standard deviation within each 5×5 block), which is high wherever the surface isn't smooth. Add one, retrain, and report recall and false alarms before and after on the same split. If a kernel adds nothing, say so: negative results are results, and a model with three useful filters is better than one with six of which three are noise.

12. Loop the four steps from section 53.2 over the examples, accumulating gradients per batch, and record the loss each epoch. It will match scikit-learn's shape (fast fall, then a long flattening) but not its numbers, because `MLPClassifier` uses Adam with its own initialization and shuffling. The purpose of the exercise is to see that "training" is that loop and nothing else; the purpose of using a library afterwards is that the library's version is faster, better initialized, and already debugged.

13. Changing `W_value` changes the *outputs* while leaving the attention weights untouched, because weights come from Q and K only. That is the division of labor worth remembering: **queries and keys decide who is listened to; values decide what is heard**. Give *was* a large value vector and every token's output moves toward it in proportion to how much attention it was already paying, which is a good way to see why a token that nobody attends to can carry any value it likes without affecting the result.

14.

```
def quantize_bits(matrix, bits):
    limit = 2 ** (bits - 1) - 1
    scale = np.abs(matrix).max() / limit
    return np.round(matrix / scale).clip(-limit - 1, limit).astype(np.float32) * scale

original = [c.copy() for c in model.coefs_]
baseline = model.predict_proba(Xte)[: , 1]
for bits in (8, 4):
    model.coefs_ = [quantize_bits(c, bits) for c in original]
    changed = model.predict_proba(Xte)[: , 1]
    flips = int(((baseline > 0.1) != (changed > 0.1)).sum())
    print(f"{bits}-bit: largest probability change {np.abs(baseline - changed).max():.4f}, "
          f"decisions flipped {flips} of {len(yte):,}, "
          f"memory {sum(c.size for c in original) * bits // 8:,} bytes")
model.coefs_ = original
```

```
8-bit: largest probability change 0.0300, decisions flipped 1 of 1,500, memory 5,232 bytes
4-bit: largest probability change 0.4221, decisions flipped 6 of 1,500, memory 2,616 bytes
```

Four-bit quantization halves the memory again and costs more decisions. For a small model on a laptop that trade is pointless; for a large language model that only fits on your GPU at 4 bits, it's the difference between running and not running, which is why 4-bit inference is common for LLMs and rare for small classifiers.

15. Expect an agreement rate in the 80s or low 90s: scratches one pixel wide and faint voids are hard to see at 32×32. Two consequences. First, **the model cannot be more accurate than the labels it learned from**, so a "94% accurate" model trained on 90%-agreement labels is partly fitting noise. Second, **measure label agreement before blaming the model**: if two inspectors disagree on 10% of parts, the project's first deliverable is a clearer definition of a defect, not a network.

16. Ask which 99%. Always predicting "good" already scores 92%, and 99% accuracy with the current defect rate would still mean missing about one defect in eight while the number sounds excellent. Redirect to the two numbers that matter: *"of the defects we produce, what share must we catch, and how many good parts can we afford to re-inspect?"* Then show the cost table from section 53.6, which converts those into a threshold. Managers accept this quickly, because it's the trade-off they already make with human inspectors.

17. Three fixes: (a) **collect night-shift images and retrain**, which addresses the cause; (b) **augment brightness and contrast during training**, which makes the model robust to the next lighting change nobody warned you about; (c) **normalize each image** before it reaches the model (subtract its own mean, divide by its own standard deviation), which removes the global lighting signal entirely. Do (c) first, because it's an hour's work and often fixes most of the gap, then (a) and (b) properly. And add the input-brightness monitor whatever you do, because the fourth lighting condition is already out there.

18. Fine-tune a pretrained model when your images look at all like natural photographs, when you have hundreds rather than tens of thousands of labeled examples, and when you can afford a GPU for an hour. You need: the labeled images, a framework (PyTorch with torchvision), the pretrained weights, and a validation set you trust. The reason it wins is **transfer**: the pretrained network already knows edges, textures, and shapes from millions of images, so your few hundred examples only have to teach it the last step. Train from scratch only when your images are nothing like natural photographs (X-rays, spectrograms, this chapter's synthetic lids) *and* you have the data volume to support it.